

LSB Master Blueprint

Version 2.1 — Revised Edition · Backtest-First Sequencing · Graphify Knowledge Layer · Claude Code Implementation Playbook

Supersedes: Master Blueprint v2.0 (50 pp) · Incorporates: Addendum v2.1 in full

Source of truth: **LSB System Requirements v2.0** (the specification remains immutable)

Date: 10 June 2026 · Johannesburg, ZA

Plan horizon: ~7.5 months to full five-stage system · GO/NO-GO at **week 7**

WHAT CHANGED FROM V2.0

Three structural revisions, no spec changes: (1) **backtest-first sequencing** — the strategy is validated on replayed data at week 7, before any broker, Docker or VPS work; (2) **walk-forward validation** replaces the single-backtest gate; (3) a **Graphify knowledge layer** turns the spec, schema and code into one queryable graph that every Claude Code session shares. Part 15 is now a complete step-by-step Claude Code playbook: every command, every session prompt, every acceptance check from an empty folder to the running five-stage system.

STANDING DISCLAIMER

This is an engineering blueprint, not investment advice. Most retail algorithmic strategies show no durable edge; this document's most important feature is the gate that gives you permission to stop. Nothing here promises profit.

Table of Contents

Part 1	Executive Summary & Founding Thesis
Part 2	Strategic Foundation — Five Stages, One Sequence
Part 3	System Architecture — Layers, Modules M1–M14, the Advisor Seam
Part 4	Broker Integration (MT5) — Decided Week 1, Built After the Gate
Part 5	Database Schema — Seven Tables, Full DDL
Part 6	Stage 1 · The Deterministic Engine & the Nine-State Machine
Part 7	Replay Harness & Walk-Forward Validation (<i>new in v2.1</i>)
Part 8	Stage 2 · Analytics & the Data Asset
Part 9	Stage 3 · The Bayesian Evidence Engine
Part 10	Stage 4 · The Quantum-Inspired Hypothesis Engine (Honest Framing)
Part 11	Stage 5 · The Autonomous Research Layer (Human-Gated)
Part 12	Deployment & Operations — Built Only After a GO
Part 13	The Compressed Roadmap — 7.5 Months, Same Gates
Part 14	The Graphify Knowledge Layer (<i>new in v2.1</i>)
Part 15	The Claude Code Implementation Playbook — Every Step
Part 16	Milestones, KPIs & Go/No-Go Gates
Part 17	Hiring Roadmap — Triggered by Revenue, Not Hope
Part 18	The Honest Risk Register
App. A	Configuration Parameter Reference
App. B	Glossary

Executive Summary & Founding Thesis

A deterministic, liquidity-sweep trading system that earns its intelligence in stages — now sequenced so the market gets to veto the project at week seven, before a cent of infrastructure is spent.

The Liquidity Strategy Bot (LSB) trades a single, fully-specified idea: institutional liquidity sweeps at structural levels, qualified through eight non-negotiable gates, sized and protected by deterministic risk rules. The company's long-term asset is not the strategy — it is the **decision warehouse**: a complete record of every evaluated candle, every qualification, every rejection and every outcome, against which increasingly intelligent layers are trained. The build proceeds in five stages, each of which must earn the right to start the next.

1.1 The Five Stages

Stage	Layer	What it adds
1	Deterministic engine	The v2.0 spec implemented literally — M1-M14, the nine-state machine, eight entry gates, hard risk limits. Boring, exact, trusted.
2	Analytics & data	Every candle and decision captured to the warehouse; expectancy sliced by context. The seed corn for everything later.
3	Bayesian evidence engine	A calibrated confidence score per qualified setup, learned from logged outcomes. Advises the entry gate; can only reduce risk.
4	Hypothesis engine	A probability vector over market regimes ("quantum-inspired" — accurately, a classical Bayesian belief state), updated each candle; entropy informs selectivity.
5	Autonomous research	A ResearchAgent that proposes — never deploys — strategy refinements; a human approval gate is enforced in code.

1.2 The v2.1 Thesis: Validate Before You Build

Version 2.0 spent its first three months building live machinery — broker bridge, Docker, VPS — before asking whether the strategy works. Version 2.1 inverts that. The deterministic engine is built **offline** against a candle-replay harness with simulated fills; a **walk-forward backtest** renders the GO/NO-GO verdict at **week 7**; only a GO releases the four weeks of live-infrastructure work. The cheapest possible failure is now seven weeks of reusable engineering and zero infrastructure spend.

THE PLAN IN ONE PARAGRAPH

Week 1: data acquired and audited, broker pattern decided, repo scaffolded, knowledge graph built. Weeks 1-5: the deterministic engine, module by module, against replayed candles, with determinism enforced in CI. Weeks 5-7: walk-forward validation → **GO/NO-GO**. Weeks 8-11 (GO only): broker execution, Docker, VPS, alerting. Weeks 12-19: an eight-week demo forward test — fixed, non-compressible — while the Bayesian layer is built in parallel against backtest logs. Weeks 20-30: confidence scoring live in shadow → advisory mode, then outcome learning and the hypothesis engine, gated on trade count. Week 30+: the human-gated research layer. Total: ~7.5 months to the full five-stage system.

1.3 Where the Moat Lives

Anyone can copy a rule set. Nobody can copy a year of your decision warehouse: the per-candle record of what the system saw, what it did, and what happened next, all reproducible via `config_hash`. Stages 3-5 convert that asset into compounding advantage. This is why decision capture (M13) is architecturally mandatory and is never the corner to cut.

1.4 The Determinism Invariant

The same candles and the same configuration must always produce **byte-identical decisions**. v2.1 enforces this in CI with golden replay fixtures (Part 7.3). Every intelligence layer attaches behind the `Advisor` interface and can be switched off, restoring the deterministic baseline exactly — intelligence that can't be turned off and compared isn't trustworthy.

Strategic Foundation — Five Stages, One Sequence

Why the order is the strategy: each stage produces the asset the next one consumes.

2.1 The Dependency Chain

- **Stage 1 → Stage 2:** the deterministic engine produces decisions worth recording only because it is exact and reproducible. Garbage decisions in, garbage training data out.
- **Stage 2 → Stage 3:** the Bayesian engine learns likelihood ratios from logged outcomes. No warehouse, no likelihoods — the math literally has no inputs.
- **Stage 3 → Stage 4:** the hypothesis engine extends per-setup confidence into a persistent belief state over regimes. It reuses the same evidence pipeline.
- **Stage 4 → Stage 5:** the research layer mines the warehouse for refinements, scores them with the same validation harness, and proposes. A human approves. Always.

2.2 Backtest-First: the v2.1 Re-Sequencing

The GO/NO-GO gate needs exactly three things: the strategy logic (M1–M8, M11–M13), historical data, and a replay harness with honest cost modelling. It does not need a broker connection, a container stack or a server. Therefore those come later — and only conditionally. This single re-ordering moves the gate from week 12 to week 7 and converts a NO-GO from a six-figure-rand disappointment into a cheap, fast answer.

THE ONE-SENTENCE VERSION

Walk the road before you build the car: backtest the strategy against replayed data with simulated fills, and only build the live machinery once the edge has earned it.

2.3 "Quantum-Inspired," Said Honestly

SAY IT PLAINLY

There is **no quantum computer** anywhere in this system, and there does not need to be. The Stage-4 hypothesis engine is a classical Bayesian state model that borrows the *vocabulary* of quantum mechanics — superposition as "hold many weighted futures," measurement as "update on evidence" — running on ordinary CPUs. Used internally as intuition, the framing is genuinely useful. Used externally to imply exotic technology or guaranteed performance, it would be misleading — to investors, customers, or yourself. Every document, pitch and README inherits this paragraph.

2.4 Principles Carried Through Every Part

Principle	Consequence in the design
The spec is immutable	Claude Code implements the v2.0 spec literally; deviations are proposals, logged and human-approved. The Graphify graph (Part 14) makes the spec queryable, not editable.
Intelligence advises, never overrides	Stages 3–5 attach behind the Advisor interface; they can reduce risk or veto, never qualify a trade the rules rejected, and never bypass M8/M11.
Evidence beats schedule	The forward test is 8 weeks minimum; outcome learning is gated on trade count, not calendar; a NO-GO means stop and rethink, not proceed.
Everything reproducible	config_hash on every row; append-only outcome tables; golden fixtures in CI; one Compose file plus one dump rebuilds the world.
Humans own high stakes	

The research layer proposes; deployment of any live change requires explicit human approval, enforced in code.

System Architecture

A layered core that evaluates every closed candle through a fixed sequence, with an intelligence rail that attaches behind interfaces and can never override the risk layer.

3.1 The Layered View



Figure 3.1 — Layered architecture. Evaluation flows upward each closed candle; the intelligence rail attaches behind the Advisor interface and advises the entry gate without ever overriding the deterministic risk layer.

3.2 Module Map (M1–M14)

Fourteen independent, separately-testable modules — the canonical list, mirroring the v2.0 specification exactly. M1–M6 sense; M7–M8 decide; M9–M10 act; M11–M12 protect; M13–M14 observe.

ID	Module	Responsibility
M1	Market Data Ingestion	Closed OHLCV candles, tick/spread data, session clock, gap detection per instrument
M2	Indicator Engine	EMA(21/50/89), ATR(14), EMA slopes, 8-way candle classification
M3	Trend Analysis	EMA-hierarchy + slope → BULLISH / BEARISH / NEUTRAL / INVALID
M4	Structure Detection	Ascending/descending triangles, compression validation, apex proximity
M5	Liquidity Zone Engine	Resistance/support block mapping, sweep detection, 0–100 five-factor sweep score
M6	Volatility Monitor	ATR state (COMPRESSED → EXTREME), candle-size analysis, spread/slippage tracking
M7	Entry Qualification	Sequential evaluation of the 8 entry conditions → QUALIFIED / REJECTED with reason
M8	Risk Computation	Position sizing, structural stop, R:R ≥ 2.5 validation, risk-tier and drawdown checks
M9	Order Execution	Broker interface; stop orders with attached SL/TP; idempotent IDs; pre-exec gate
M10	Trade Management	Breakeven at +1R, EMA21 trail from +1.5R, defensive exits
M11	Risk Protection	Daily/weekly drawdown halts, loss-streak tracking, emergency stop
M12	Session & News Filter	Session validity, Tier-1 news windows, low-liquidity rejection
M13	Logging & Analytics	Writes every warehouse row (Part 5); metrics; full audit trail — <i>the data asset</i>
M14	Alerting	Status, trade alerts, halt notifications, error reporting (Telegram/email)

3.3 The Per-Candle Evaluation Sequence

On every closed H1 candle, per instrument, in fixed order: M1 ingest → M2 indicators → M3 trend → M4 structure → M5 liquidity/sweep → M6 volatility → M12 session/news → M7 qualification (8 gates, short-circuit on first failure, reason recorded) → M8 risk computation → [Advisor consulted, Stage 3+] → M9 order (live only) → M13 snapshot row written *always*, including NO_SETUP candles. M10 and M11 run continuously alongside.

3.4 The Advisor Seam

PYTHON • THE ONLY DOOR INTELLIGENCE MAY USE

```
class Advisor(Protocol):
    def assess(self, setup: QualifiedSetup, ctx: MarketContext) -> Assessment:
        """Returns confidence in [0,1] and an optional veto.
        MAY reduce position size or veto a qualified setup.
        MUST NOT create, enlarge, or re-qualify anything.
        MUST be side-effect-free and fast (<50ms)."""

class NullAdvisor:          # Stage 1-2: the permanent baseline
    def assess(self, setup, ctx): return Assessment(confidence=1.0, veto=False)
```

Every intelligence stage implements this interface. Switching back to `NullAdvisor` restores the deterministic baseline byte-for-byte — the determinism invariant in code.

3.5 Repository Structure

```
lsb/
├─ CLAUDE.md                # governing instructions for Claude Code (Part 15.2)
├─ docs/                    # v2.0 spec + this blueprint → Graphify corpus
├─ graphify-out/            # committed knowledge graph (Part 14)
├─ config/                  # instrument configs, hashed into config_version
├─ src/lsb/
│   ├─ core/                # state machine, event bus, clock
│   ├─ modules/m01_data ... m14_alerts/
│   ├─ advisor/             # NullAdvisor, bayesian/, hypothesis/
│   ├─ replay/              # harness, fill simulator, walk-forward (Part 7)
│   └─ broker/              # MT5 adapter behind the M9 interface (Part 4)
├─ tests/
│   ├─ unit/                # per-module, hand-verified reference values
│   ├─ fixtures/golden/     # input candles + expected decision logs
│   └─ adversarial/         # forced drawdowns must trigger halts
├─ migrations/              # SQL DDL, ordered
└─ docker/                  # built in Phase 4, not before
```

Broker Integration (MT5)

The decision is made in week 1; the build happens after the gate. The core never knows which bridge it's talking to.

4.1 The Three Bridge Patterns

Pattern	How it works	Verdict
Native Windows (default)	Windows VPS runs MT5 terminal + the official MetaTrader5 Python package in-process	Recommended. Most reliable, best-documented path; modest extra VPS cost.
Wine on Linux	MT5 under Wine in a container	Fragile under terminal updates; acceptable for experiments, not for live capital.
EA socket bridge	A thin MQL5 Expert Advisor exposes a socket/REST bridge; core stays on Linux	Fallback. More moving parts but clean separation; use if Windows licensing becomes a constraint.

V2.1 CHANGE (FIX F4)

This choice is a **week-1 decision**, recorded in [docs/decisions/ADR-001-broker-bridge.md](#) — but the implementation is **Phase-4 work**, after a GO. Until then M9 talks to the replay harness's simulated broker, which implements the same interface.

4.2 Reliability Contract (Phase 4 acceptance)

- **Heartbeat:** terminal connectivity checked every cycle; loss → alert + ORDER-SAFE mode (manage open trades, no new entries).
- **Reconciliation:** on every restart, open positions are reconciled against the DB; a crash never orphans a live trade.
- **Idempotency:** every order carries a deterministic client ID; retries can never double-fill.
- **Deriv synthetics note:** Boom/Crash spike behaviour gets per-instrument configs and distinct ATR-EXTREME thresholds (Appendix A); never share parameters across synthetic and conventional instruments.

Database Schema — Seven Tables, Full DDL

Capturing decisions — not just trades — is the whole thesis. market_snapshot is the spine; every trade and hypothesis points back to the exact candle that produced it.

5.1 The Observation Table

One row per evaluated candle per instrument — including candles where nothing happened. At a handful of instruments on H1 this is thousands of rows a month, trivial for PostgreSQL, and it is the seed corn for every model you will ever train. Design partition-ready on `ts`; adopt monthly range partitioning only when instrument count or timeframe demands it.

SQL • MARKET_SNAPSHOT

```
CREATE TABLE market_snapshot (
  snapshot_id BIGSERIAL PRIMARY KEY,
  ts          TIMESTAMPTZ NOT NULL,          -- candle close, UTC
  instrument  TEXT NOT NULL,
  timeframe   TEXT NOT NULL,                -- 'H1' | 'H4' | 'D1'
  open NUMERIC(18,8), high NUMERIC(18,8), low NUMERIC(18,8), close NUMERIC(18,8),
  volume      NUMERIC(18,4),
  ema21 NUMERIC(18,8), ema50 NUMERIC(18,8), ema89 NUMERIC(18,8),
  atr         NUMERIC(18,8),
  atr_state   TEXT,                        -- COMPRESSED|NORMAL|ELEVATED|EXTREME
  trend_state TEXT,                        -- BULLISH|BEARISH|NEUTRAL|INVALID
  structure_state TEXT,                   -- ASC_TRIANGLE|DESC_TRIANGLE|NONE
  candle_type TEXT,                       -- 8-way classification
  sweep_score SMALLINT,                   -- 0..100, NULL if no sweep
  session     TEXT,                       -- LONDON|NY|OVERLAP|ASIA|OFF
  trade_decision TEXT NOT NULL,           -- TRADED|REJECTED|NO_SETUP
  config_hash TEXT NOT NULL REFERENCES config_version,
  UNIQUE (instrument, timeframe, ts)
);
CREATE INDEX ix_snap_inst_ts ON market_snapshot (instrument, ts DESC);
CREATE INDEX ix_snap_decision ON market_snapshot (trade_decision)
  WHERE trade_decision <> 'NO_SETUP'; -- fast access to the interesting rows
```

5.2 The Trade Lifecycle

A trade moves through three immutable rows — `trade_signal` (M7's verdict, written even on rejection), `trade_execution` (only if filled), `trade_outcome` (on close) — so you can study the gap between intention and fill.

SQL • TRADE_LIFECYCLE

```

CREATE TABLE trade_signal (
  trade_id          UUID PRIMARY KEY,
  snapshot_id       BIGINT NOT NULL REFERENCES market_snapshot,
  ts_signal         TIMESTAMPTZ NOT NULL,
  instrument        TEXT NOT NULL,
  direction         TEXT,                -- LONG | SHORT
  sweep_score       SMALLINT,
  rr_planned        NUMERIC(6,2),
  qualified         BOOLEAN NOT NULL,
  rejection_reason  TEXT,                -- which of the 8 gates failed
  config_hash       TEXT NOT NULL REFERENCES config_version
);

CREATE TABLE trade_execution (
  execution_id      BIGSERIAL PRIMARY KEY,
  trade_id          UUID NOT NULL REFERENCES trade_signal,
  ts_filled         TIMESTAMPTZ NOT NULL,
  entry_price       NUMERIC(18,8), sl_price NUMERIC(18,8), tp_price NUMERIC(18,8),
  size_lots         NUMERIC(10,2),
  spread_at_fill    NUMERIC(10,5), slippage NUMERIC(10,5),
  broker_order_id  TEXT UNIQUE           -- idempotency anchor
);

CREATE TABLE trade_outcome (                -- append-only; corrections supersede, never edit
  outcome_id        BIGSERIAL PRIMARY KEY,
  trade_id          UUID NOT NULL REFERENCES trade_signal,
  ts_closed         TIMESTAMPTZ NOT NULL,
  exit_price        NUMERIC(18,8),
  exit_reason       TEXT,                  -- TP|SL|TRAIL|DEFENSIVE|HALT
  r_multiple        NUMERIC(8,4) NOT NULL, -- THE number the learning loop trains on
  pnl_account       NUMERIC(14,2),
  supersedes        BIGINT REFERENCES trade_outcome
);

```

THE ONE NORMALISED NUMBER

`r_multiple` — outcome expressed in initial-risk units — is what every learning loop trains on. The whole intelligence programme hinges on this one field being recorded correctly, for every close, forever.

5.3 Intelligence & Audit Tables

SQL • INTELLIGENCE & AUDIT

```

CREATE TABLE hypothesis_snapshot (
  hypothesis_id BIGSERIAL PRIMARY KEY,
  snapshot_id  BIGINT NOT NULL REFERENCES market_snapshot,
  state_vector JSONB,           -- {"bearish_continuation":0.35, ...} sums to 1.0
  dominant_state TEXT,          -- argmax, for fast filtering
  entropy      NUMERIC(6,4)     -- uncertainty of the vector
);

CREATE TABLE state_transition (
  transition_id BIGSERIAL PRIMARY KEY,
  ts            TIMESTAMPTZ NOT NULL,
  from_state    TEXT NOT NULL,
  to_state      TEXT NOT NULL,
  reason        TEXT,           -- e.g. 'DAILY_DRAWDOWN_BREACH'
  instrument     TEXT
);

CREATE TABLE config_version (
  config_hash  TEXT PRIMARY KEY,      -- sha256 of canonicalised params
  params       JSONB NOT NULL,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

Seven tables total. `hypothesis_snapshot` stays empty until Stage 4, but exists from day one — no migration later.

`state_transition` is the audit trail that answers "why did the bot halt at 14:00 on Tuesday?" with certainty. The spec's 25-field trade journal (§14.1) is a view joining signal × execution × outcome × snapshot — derived, never duplicated.

Stage 1 • The Deterministic Engine

The whole specification, implemented literally — no optimisation, no learning, no creativity. Boring, exact, trusted.

Everything intelligent the company will ever build sits on this layer. The governing instruction to whoever (or whatever) writes it: treat the specification as immutable, implement each module independently with full unit tests, and make the initial release entirely rule-based. Intelligence bolts on from the side later — it never edits this core.

6.1 The Eight Entry Gates (M7)

Evaluated sequentially; the first failure rejects the setup and records the reason. All eight must pass.

#	Gate	Pass condition (per spec; parameters in Appendix A)
1	Trend alignment	M3 state matches trade direction; EMA hierarchy and slope valid
2	Structure present	M4 reports a valid triangle compression with acceptable apex proximity
3	Liquidity sweep confirmed	M5 detects a sweep of a mapped block in the qualifying direction
4	Sweep quality	Five-factor sweep score $\geq$ configured threshold
5	Candle confirmation	Closing candle classification confirms rejection/continuation per spec
6	Volatility acceptable	M6 ATR state within allowed set; spread within limit
7	Session & news clear	M12: valid session, no Tier-1 news window, liquidity adequate
8	Risk viable	M8: structural stop placeable, R:R $\geq$ 2.5, drawdown headroom, tier limits OK

6.2 The Nine-State Operating Machine

The bot is always in exactly one of nine states; every transition is logged to `state_transition` with its reason.

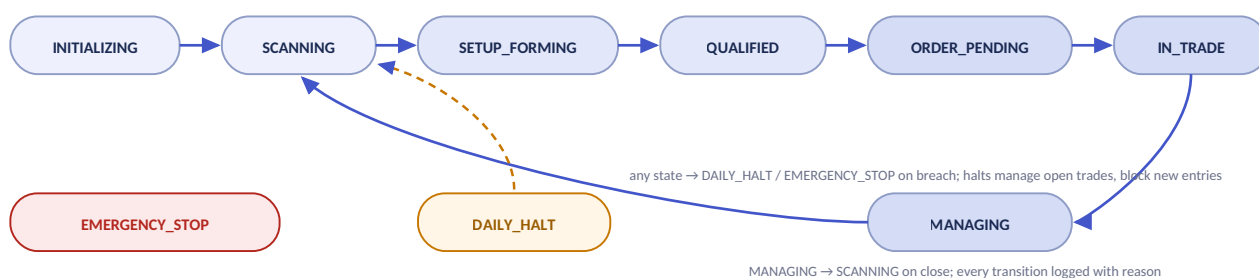


Figure 6.1 — The operating state machine (simplified view; transitions per spec). Risk states are reachable from anywhere; recovery to SCANNING requires the reset condition, never discretion.

6.3 Module Build Order (Weeks 2–5)

Bottom-up, in parallel lanes where dependencies allow; each week closes only when its modules pass tests against hand-checked fixtures.

Week	Modules	Deliverable & test gate
Week 2	M1, M2 M13	Clean candle feed; EMA & ATR match a reference spreadsheet to 8 dp; candle types correct on a labelled fixture set; snapshot writes wired from day one
Week 3	M3, M4 M6, M12	Trend state matches hand-labelled charts; triangle detector finds known patterns, rejects noise; ATR states correct at every boundary; session/news filter on fixtures

Week 4	M5	Block identification + sweep detection; the 5-factor score reproduces worked examples exactly
Week 5	M7, M8, M10, M11	All 8 gates wired in sequence; sizing & structural stops correct; halts fire at thresholds (adversarial tests); full setup → order → manage → exit on replayed history via the simulated broker

TESTING NON-NEGOTIABLES

(1) Every numeric module is tested against **hand-verified reference values**, not just "it runs." (2) The full pipeline runs on **replayed historical candles** with known outcomes before any live connection. (3) Risk logic gets **adversarial tests** — force a daily-loss breach and assert the bot enters DAILY_HALT and refuses entries. (4) **Determinism is tested in CI**: same candles + config → byte-identical decision log (Part 7.3).

Replay Harness & Walk-Forward Validation

The machinery that lets the market veto the project at week seven — and the fixtures that make determinism a CI property instead of a promise.

7.1 The Replay Harness (Week 1–2 infrastructure)

The harness feeds historical candles through the identical per-candle sequence the live system uses — same modules, same state machine, same M13 writes (to a backtest-tagged schema) — with a **simulated broker** behind the M9 interface. One code path, two clocks. Fill simulation is deliberately pessimistic:

- **Spread:** historical spread where available, otherwise a conservative per-instrument constant (Appendix A).
- **Slippage:** entry stops fill at trigger + modelled slippage scaled by ATR state; EXTREME state doubles it.
- **Costs:** commission and swap modelled per instrument. A thin edge that costs erase is not an edge.
- **Gaps:** if price gaps through a stop, the fill is the gap open, not the stop price — the honest case.

7.2 Walk-Forward Validation (fix F2)

A single backtest over one window with one parameter set is the classic overfitting trap. The gate therefore runs **rolling train/test windows**: anchor 18 months, test the next 6, roll forward 3, across the full 3-year corpus and ≥ 3 instruments. The verdict is rendered on the **concatenated out-of-sample segments only**.

Gate criterion (G3, week 7)	Bar
Out-of-sample expectancy	Positive in aggregate <i>and</i> in a majority of test windows, per instrument
Profit factor (OOS, after costs)	Meets the spec §13 threshold on the concatenated OOS set
Max drawdown (OOS)	Within the spec §13 limit; no single window catastrophically worse
Sample size	Enough qualified trades across OOS windows for the verdict to mean something (spec §13 minimum)
Parameter sensitivity	$\pm 20\%$ perturbation of key thresholds does not flip the verdict — an edge that exists only at one magic number is noise

THE SPEC STAYS CANONICAL

The numeric thresholds live in System Requirements v2.0 §13 and Appendix A — this document deliberately does not fork them. What v2.1 changes is *how* they are tested: out-of-sample, rolling, cost-loaded, sensitivity-checked.

7.3 Golden Fixtures — Determinism in CI (fix F5)

`tests/fixtures/golden/` holds curated candle sets with committed expected outputs: the full decision log, every state transition, every signal row. CI replays each fixture and fails on any byte difference. Property-based tests cover M8's arithmetic (sizing monotonic in risk; stops always structural; R:R math exact). Any intentional behaviour change must regenerate fixtures in a reviewed commit — which is precisely the audit moment you want.

7.4 Anti-Overfitting Discipline (carried into Stages 3–5)

- Minimum sample sizes before any likelihood is trusted; Bayesian shrinkage toward the prior below them.
- Mandatory out-of-sample checks for every learned weight; improvement shown on held-out trades, or the weight is discarded.
- Research-layer proposals are scored with this same walk-forward harness — one validator, every stage.

Stage 2 • Analytics & the Data Asset

If it isn't captured, it never happened. The warehouse is the company.

8.1 Capture Discipline

M13 writes a `market_snapshot` row for **100% of evaluated candles** — TRADED, REJECTED and NO_SETUP alike — because rejections are exactly the data that teaches Stage 3 what a near-miss looks like. Every row carries `config_hash`, so backtest-vs-live comparisons are honest and any historical decision is reconstructable years later.

8.2 The Dashboard Questions

Grafana (Phase 4+) answers, at a glance: equity curve and drawdown vs halts; expectancy sliced by instrument, session, ATR state and sweep-score tier; rejection reasons ranked (which gate fails most, and is that the spec working or the spec wrong?); live-vs-backtest divergence on identical configs — the single most important health metric the system has.

8.3 During the Offline Phase

v2.1 runs Stage-2 capture from week 2 — the replay harness writes the same rows. By the time the gate arrives, the analytics queries are already debugged, and the GO/NO-GO review reads from the same dashboard the live system will use.

Stage 3 • The Bayesian Evidence Engine

A calibrated confidence score for every qualified setup, learned from the warehouse — attached as an advisor, never a gatekeeper.

9.1 The Math, Plainly

For each contextual feature f (sweep-score tier, ATR state, session, trend strength, structure quality...), the warehouse yields a **likelihood ratio** $LR_f = P(f \mid \text{win}) \div P(f \mid \text{loss})$. Evidence combines in log-odds:

```
log-odds(win | setup) = log-odds(prior) +  $\sum_f \log LR_f$   
confidence = sigmoid(log-odds) → calibrated to [0,1]
```

Counts are Laplace-smoothed; features below minimum sample size shrink toward $LR = 1$ (no evidence). Calibration — a "70%" signal winning ~70% of the time — is verified on held-out data before the score is shown to anyone, let alone wired to sizing.

9.2 Deployment Ladder

Mode	Behaviour
Offline (wk 12–19)	Trained and evaluated against backtest + forward-test logs; never touches live decisions
Shadow (wk 20–21)	Scores every live qualified setup; logged, compared, invisible to M7/M8
Advisory (wk 22+)	Implements Advisor : may scale size down or veto within configured bounds; can never qualify, enlarge, or bypass M8/M11

Per-factor contributions are logged with every assessment, so any veto is explainable after the fact: "rejected at 0.31 confidence, dominated by ASIA-session LR 0.4."

Stage 4 • The Quantum-Inspired Hypothesis Engine

A probability vector over market regimes, updated each candle. Quantum vocabulary, classical math — and we say so.

10.1 The MarketStateVector

A distribution over mutually-exclusive regime hypotheses — e.g. `{bullish_continuation, bearish_continuation, range_compression, sweep_reversal_up, sweep_reversal_down}` — summing to 1.0, persisted to `hypothesis_snapshot` every candle with its **entropy**. Each candle's evidence (trend, structure, sweep, volatility deltas) updates the vector by Bayes' rule; regime transitions apply a learned transition matrix — mathematically a hidden-Markov-style belief update, "superposition" only as metaphor.

10.2 How It Advises

- **Selectivity:** high entropy (the market "undecided") tightens the advisor — confidence is scaled down when no hypothesis dominates.
- **Alignment:** a qualified setup that contradicts the dominant hypothesis gets reduced size; one aligned with it passes untouched. Never enlarged.
- **Research fuel:** the logged vectors become features for Stage 5 — "does entropy at entry predict R-multiple?" is a SQL query, not a guess.

FRAMING RULE

Internal docs may use the quantum metaphor freely; anything external describes this as a **Bayesian belief-state model over market regimes**. No exotic hardware, no speed-up claims, ever.

Stage 5 • The Autonomous Research Layer

A ResearchAgent that mines the warehouse and proposes refinements. It can write reports; it cannot touch production. Ever.

11.1 The Loop

Scheduled runs mine the warehouse for candidate refinements (threshold shifts, session exclusions, feature ideas), score each with the Part-7 walk-forward harness, and emit a **proposal document**: hypothesis, evidence, OOS results, risk notes, and the exact config diff. Proposals land in a review queue.

11.2 The Human Gate — Enforced in Code

- The agent's credentials can write to `proposals/` and read the warehouse. They **cannot** write to `config/`, the database config tables, or production deployment — enforced by permissions, not policy.
- Applying a proposal requires a human-signed config change, which mints a new `config_hash` — the audit trail is automatic.
- Every proposal, approved or rejected, is retained: the rejection history is itself research data.

Deployment & Operations — Built Only After a GO

Four weeks of work, released by the gate. One Compose file plus one database dump rebuilds the world.

12.1 Topology

Docker Compose on a single VPS: the LSB core, PostgreSQL, Grafana, and the dashboard. Broker side per the ADR-001 decision (Part 4) — default a Windows VPS running MT5 + the official Python package, with the core either co-located or talking across the bridge. Only the two UI ports are exposed; the database lives on the private Docker network; the host runs a firewall and automatic security updates.

12.2 Operations Contract

Concern	Practice
Backups	Nightly encrypted <code>pg_dump</code> to off-site object storage. The warehouse is the one truly unrecoverable loss.
Disaster recovery	Whole stack = one Compose file + one dump; recovery rehearsed on a fresh box periodically. An untested backup is a hope, not a plan.
State recovery	On restart the core reconciles open positions against the DB and resumes in the correct state; a crash never orphans a trade.
Secrets	Credentials in environment/secrets files outside the repo, never committed; UIs behind auth + TLS.
Alerting (M14)	Halts, connectivity loss, emergency stops and abnormal drawdown push immediately (Telegram/email). A halt the founder doesn't hear about defeats its purpose.

12.3 What It Costs

€6–15/month for a Contabo-class VPS (plus a modest premium if the Windows broker VPS is separate), ~€1/month off-site backup, €0 software — all open-source. The expensive resource is not money; it is disciplined engineering time and the patience to let evidence accumulate.

The Compressed Roadmap — 7.5 Months, Same Gates

Twelve months become roughly seven and a half by re-sequencing and parallelism — never by cutting tests, shortening the forward test, or softening a gate.

13.1 Phase Table

#	Phase	Timeline	Exit criterion
0	Data & decisions	Week 1	3y × ≥3 instruments of audited H1 data on disk; ADR-001 broker decision recorded; repo scaffolded; Graphify installed, graph v1 built from the spec corpus.
1	Foundation + replay harness	Weeks 1–2	Config system (hash-versioned), schema migrated, logging, state-machine skeleton, replay harness with simulated fills. No Docker, no VPS, no broker.
2	Deterministic engine (offline)	Weeks 2–5	M1–M8, M10–M13 to spec, unit-tested; golden fixtures green in CI; full setup→order→manage→exit on replayed data.
3	Strategy validation	Weeks 5–7	GO/NO-GO (G3). Walk-forward backtest meets spec §13 thresholds out-of-sample across ≥3 instruments / 3 years; sensitivity check passes.
4	Live infrastructure	Weeks 8–11	<i>Only after GO.</i> M9 live execution per ADR-001, M14 alerting, Docker stack, VPS deploy, crash-recovery & reconciliation tests passing.
5	Demo forward test Bayesian dev	Weeks 12–19	8 weeks demo trading (fixed, non-compressible); live-vs-backtest divergence within tolerance. In parallel: Bayesian engine trained offline, calibration verified on held-out data.
6	Bayesian advisor live	Weeks 20–24	Shadow → advisory; calibration re-verified on forward-test outcomes; never overrides M8/M11.
7	Outcome learning + hypothesis engine	Weeks 24–30	Gated on trade count, not calendar : weights regenerated from real outcomes, improvement shown held-out; state vector logged each candle; entropy informs selectivity.
8	Autonomous research layer	Week 30+	ResearchAgent produces reviewed proposals; the human gate enforced by permissions.

Gate timing: v2.0 vs v2.1

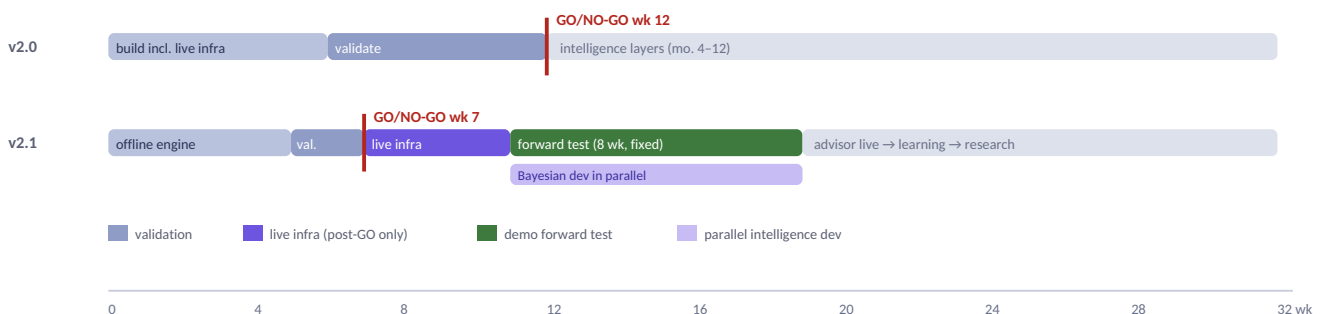


Figure 13.1 — The gate moves five weeks earlier because nothing it needs depends on live infrastructure; Bayesian development rides alongside the fixed forward test instead of waiting for it.

13.2 What Cannot Be Compressed

THE HONEST FLOOR

Three things are evidence, not engineering: (1) the **8-week forward test** — fewer weeks means fewer regimes sampled and a result you cannot trust; (2) **outcome learning**, gated on a statistically meaningful trade count; (3) **the gate itself** — a week-7 NO-GO means stop and rethink, not proceed on schedule. A 7.5-month plan that quietly trims these is a faster route to an untrustworthy system, which is slower than a 12-month plan that works.

The Graphify Knowledge Layer

The spec, the schema and the code become one queryable graph — committed to the repo, shared by every Claude Code session and every future hire.

14.1 What and Why

Graphify (open-source, MIT) parses a project folder — code, Markdown, PDFs, SQL — into a typed knowledge graph in `graphify-out/`. Code is parsed locally via tree-sitter with no LLM calls and no source leaving the machine; queries traverse the graph instead of re-reading raw files; the graph doubles as an MCP server and registers as a Claude Code skill. Verified working at v0.8.36 (PyPI package `graphifyy` — double y).

- **Spec-as-graph:** seeding `/docs` with this blueprint and the v2.0 spec makes design rationale queryable and linked to implementing code — a referee for the "spec is immutable" rule.
- **Schema introspection:** `--postgres` wires the seven warehouse tables into the graph as nodes connected to the modules that touch them.
- **Blast radius:** `graphify affected "M8"` answers "what does changing risk math touch?" structurally, before the change.
- **Shared by design:** the committed graph travels with the repo — zero-setup shared context for every session and every hire.

14.2 Operating Rhythm

When	Practice
Week 1	Install; build graph v1 from <code>/docs</code> + scaffold; commit <code>graphify-out/</code> .
During build	<code>graphify watch .</code> in a side terminal; every Claude Code session starts from a graph query, not a document re-read.
After each merge	<code>graphify update .</code> — local AST re-extraction, no LLM; <code>--force</code> after deletions.
Every phase exit	Rebuild, re-cluster, commit; the graph diff is a cheap structural review of what the phase changed.
Phase 4+	Add <code>--postgres</code> introspection of the live warehouse.
Hiring milestones	Onboarding = the committed graph + <code>graphify explain</code> , not a 50-page read.

HONEST CAVEAT

Graphify accelerates the **build** — fewer tokens per query, faster onboarding, structural impact answers. It contributes nothing to the **edge**. Use `--no-label` to keep community naming fully local; treat the graph as derived state — spec and code remain the source of truth.

The Claude Code Implementation Playbook

Every step from an empty folder to the running five-stage system: setup, governing files, and a numbered session prompt for each phase with its acceptance check. Work top to bottom; never start a session whose predecessor's check hasn't passed.

15.1 Step 0 — One-Time Setup

TERMINAL • WORKSTATION

```
# 1. Claude Code (requires Node 18+; see docs.claude.com for current install)
npm install -g @anthropic-ai/claude-code

# 2. Project root + git
mkdir lsb && cd lsb && git init

# 3. Python toolchain
python3 -m venv .venv && source .venv/bin/activate
pip install pytest hypothesis psycpg2-binary pandas

# 4. Graphify (note: package is graphifyy, double y)
pip install graphifyy
graphify install --platform claude

# 5. Seed the knowledge corpus
mkdir -p docs && cp ~/LSB_System_Requirements_v2.0.docx docs/
cp ~/LSB_Master_Blueprint_v2.1.pdf docs/

# 6. Local PostgreSQL for dev (Docker is allowed locally; the *VPS* waits for the gate)
docker run -d --name lsb-pg -e POSTGRES_PASSWORD=dev -p 5432:5432 postgres:16

# 7. Launch
claude
```

15.2 Step 1 — The Governing File: CLAUDE.md

Created before any code; every session inherits it. Paste verbatim into `CLAUDE.md` at the repo root:

```
# LSB — Governing Instructions
1. The specification (docs/LSB_System_Requirements_v2.0) is IMMUTABLE.
   Implement it literally. Any deviation is a written proposal in
   docs/proposals/, never silent code.
2. Before reading source files, query the knowledge graph:
   `graphify query "... "` / `graphify affected "...`. Re-read raw files
   only when the graph is insufficient.
3. Every module (M1–M14) is built independently with unit tests against
   hand-verified reference values BEFORE integration.
4. Determinism is law: same candles + same config → byte-identical
   decisions. Never introduce wall-clock time, randomness, dict-ordering
   dependence, or float nondeterminism into a decision path.
5. The initial release is entirely rule-based. Intelligence lives only
   behind the Advisor interface and may only reduce risk or veto.
6. Risk code (M8, M11) gets adversarial tests. Forced breaches MUST halt.
7. Run `pytest -q` and the golden-fixture replay before declaring any
   task complete. A task with red tests is not done.
8. After completing a task: `graphify update .` and include graphify-out/
   in the commit.
9. Never touch live-broker code paths before Phase 4 is unlocked in
   docs/PHASE_STATUS.md.
```

15.3 Step 2 — The Master Prompt (first session)

SESSION 0 • scaffold. "Read CLAUDE.md and docs/. Then: (a) create the repository structure exactly as in Blueprint Part 3.5; (b) write docs/PHASE_STATUS.md marking Phase 0 active and Phases 4+ locked; (c) write docs/decisions/ADR-001-broker-bridge.md recommending native-Windows MT5 with the EA-socket fallback, status: accepted; (d) set up pytest, a GitHub Actions (or local) CI script running unit tests + golden-fixture replay; (e) run /graphify . and commit everything as 'scaffold + knowledge graph v1'."

Accept when: tree matches Part 3.5; CI runs green (zero tests is fine today); `graphify-out/graph.json` exists and `graphify query "entry qualification"` returns spec content.

15.4 Phase 0–1 Sessions (Weeks 1–2)

SESSION 1 • data. "Write scripts/fetch_history.py to pull ≥3 years of H1 OHLCV for the Appendix-A instrument list (broker export or Dukascopy), store as Parquet under data/history/, then write scripts/audit_history.py reporting gaps, duplicate timestamps, weekend bars, DST anomalies and spread outliers. Fail the audit on any unexplained gap > 2 candles. Produce docs/data_audit_report.md."

Accept when: the audit report exists and every flagged anomaly has a written disposition.

SESSION 2 • config & schema. "Implement the config system: YAML per instrument in config/, loaded into a frozen dataclass, canonicalised and sha256-hashed to config_hash; insert-or-get into config_version. Then write migrations/001_core.sql containing the seven tables exactly as Blueprint Part 5, apply to the dev database, and write round-trip tests."

Accept when: identical configs always produce identical hashes; all seven tables exist; round-trip tests green.

SESSION 3 • state machine + event loop. "Implement the nine-state operating machine per spec, with every transition writing a state_transition row with reason; an injectable Clock (replay or live); and the per-candle evaluation loop skeleton from Blueprint Part 3.3 with module stubs. Unit-test all legal and illegal transitions."

SESSION 4 • replay harness. "Implement src/lsb/replay/: a candle replayer streaming Parquet history through the evaluation loop, and a SimulatedBroker implementing the M9 interface with the pessimistic fill model of Blueprint Part 7.1 (spread, ATR-scaled slippage, gap-through-stop fills, commission/swap). Write its unit tests from worked examples."

Accept (Phase 1 exit / Gate G1): replaying a sample month produces snapshot rows for 100% of candles, all NO_SETUP (modules are stubs), deterministically — two runs, identical output hash.

15.5 Phase 2 Sessions — The Engine (Weeks 2–5)

One session per line; lanes marked || may run as parallel sessions/worktrees. Every module session uses this template:

MODULE TEMPLATE. "Query the graph for <module>'s spec section and its neighbours. Implement src/lsb/modules/<module>/ exactly to spec — no improvements. Write unit tests against the hand-verified reference values in tests/reference/<module>.csv (create it first from worked examples and have me confirm the numbers). Run the full suite; run graphify update .; commit."

Week	Sessions	Acceptance check
Week 2	5: M1 6: M2 7: M13	EMA/ATR match reference to 8 dp; candle classifier exact on the labelled fixture set; snapshots written for every candle
Week 3	8: M3 9: M4 10: M6 11: M12	Trend states match hand-labelled charts; triangles found on known patterns, rejected on noise; ATR states exact at boundaries; session/news filter correct on fixtures
Week 4	12: M5	Sweep detection on fixtures; the 5-factor 0–100 score reproduces every worked example exactly
Week 5	13: M7 → 14: M8 → 15: M10 16: M11	8 gates short-circuit with reasons; sizing/stops/R:R property-tested; breakeven +1R and EMA21 trail from +1.5R on replay; adversarial drawdown tests force DAILY_HALT

SESSION 17 • golden fixtures (Gate G2). "Curate 5 fixture sets under tests/fixtures/golden/ — a trending month, a ranging month, a halt-triggering sequence, a gap sequence, and a known full trade lifecycle. Record expected decision logs. Wire CI to replay all five and fail on any byte difference. Document the regeneration procedure in docs/fixtures.md."

Accept (Phase 2 exit): all 14 implemented modules green; golden fixtures green; end-to-end replay produces a complete trade lifecycle.

15.6 Phase 3 Sessions — The Verdict (Weeks 5–7)

SESSION 18 • walk-forward. "Implement src/lsb/replay/walkforward.py per Blueprint Part 7.2: 18-month train / 6-month test windows rolling by 3 months across all instruments; aggregate OOS-only metrics (expectancy, profit factor after costs, max DD, trade count per window); add the $\pm 20\%$ parameter-sensitivity sweep over the Appendix-A key thresholds. Emit docs/validation_report.md with per-window tables."

SESSION 19 • the gate. "Run the full walk-forward on the audited corpus. Fill in the Gate G3 checklist in docs/PHASE_STATUS.md against spec §13 thresholds. Render the verdict. If NO-GO: produce a diagnosis — which gate rejects most, where expectancy dies, what the sensitivity sweep shows — and STOP. Do not unlock Phase 4."

THE FOUNDER'S JOB AT WEEK 7

Claude Code computes; you decide. A GO unlocks Phase 4 by editing PHASE_STATUS.md yourself — deliberately a human act. A NO-GO means the honest response is to stop and rethink the strategy, not to soften a threshold until the verdict flips.

15.7 Phase 4 Sessions — Live Infrastructure (Weeks 8–11, GO only)

SESSION 20 • broker adapter. "Phase 4 is unlocked. Implement the ADR-001 bridge behind the M9 interface: connection lifecycle, heartbeat → ORDER-SAFE mode on loss, idempotent client order IDs, stop orders with attached SL/TP, restart reconciliation against the DB. Integration-test against a demo account."

SESSION 21 • alerting. "Implement M14: Telegram + email for halts, connectivity loss, emergency stop, abnormal drawdown, and a daily heartbeat summary. Test each path."

SESSION 22 • containers & deploy. "Write docker/compose.yml (core, PostgreSQL, Grafana, dashboard; only UI ports exposed; DB on the private network), the nightly encrypted pg_dump backup job, and docs/runbook.md covering deploy, restore-from-dump rehearsal, and crash recovery. Provision the VPS per runbook; prove a kill -9 mid-trade recovers to the correct state with no orphaned position."

Accept (Phase 4 exit): demo order round-trips; kill-test passes; restore rehearsal performed once for real.

15.8 Phase 5–8 Sessions (Weeks 12–30+)

SESSION 23 • forward test start. "Start the 8-week demo forward test per runbook. Build the Grafana live-vs-backtest divergence panel; alert when divergence exceeds tolerance. No code changes to decision paths during the test except critical fixes, each regenerating fixtures in a reviewed commit."

SESSION 24 • Bayesian offline (parallel). "Implement src/lsb/advisor/bayesian/ per Blueprint Part 9: per-feature likelihood ratios with Laplace smoothing and minimum-sample shrinkage, log-odds combination, sigmoid calibration. Train on backtest decision logs; verify calibration on held-out data; emit docs/calibration_report.md with reliability curves."

SESSION 25 • shadow mode. "Wire BayesianAdvisor in shadow: score every live qualified setup, log per-factor contributions, zero influence. After 2 weeks, compare shadow scores to outcomes."

SESSION 26 • advisory mode. "Promote to advisory within Appendix-A bounds: size scaling and veto only. Prove the NullAdvisor toggle restores byte-identical baseline decisions on a replay."

SESSION 27 • outcome learning. "Implement the weight-regeneration loop reading real trade_outcome rows (gated: minimum N trades per Appendix A). Show improvement on held-out trades or discard. Human-approved config change mints the new config_hash."

SESSION 28 • hypothesis engine. "Implement src/lsb/advisor/hypothesis/ per Blueprint Part 10: the regime state vector, Bayes update per candle, transition matrix, entropy; persist hypothesis_snapshot every candle; entropy-scaled confidence in the advisor. Backtest the combined advisor against Stage-3-only."

SESSION 29 • research layer. "Implement the ResearchAgent: scheduled warehouse mining, candidate scoring through the walk-forward harness, proposal documents with exact config diffs into proposals/. Create the restricted credentials (read warehouse, write proposals/ only) and prove by test that the agent cannot write config or production."

Accept (system complete): all five stages live; every intelligence layer individually toggleable back to the deterministic baseline; the human gate enforced by permissions and proven by a failing write test.

15.9 The Session Discipline

- **One session, one objective**, ending in green tests and a commit including `graphify-out/`.
- **Start every session** with "read CLAUDE.md; query the graph for <today's target>" — never with a raw file dump.
- **Confirm reference values yourself.** Claude Code drafts `tests/reference/*.csv`; you verify the numbers before they become law.
- **Before touching risk code:** `graphify affected "M8" --depth 2` and read the blast radius aloud in the session.
- **Phase locks are yours.** Only a human edits PHASE_STATUS.md; CLAUDE.md forbids Claude Code from crossing a locked phase.

Milestones, KPIs & Go/No-Go Gates

Two different questions, never conflated: did we build it (engineering, in our control) and does it work (strategy, the market's call).

16.1 Build KPIs

Indicator	Target
Modules implemented & unit-tested	14/14 by end of Phase 2 (M9 live paths complete Phase 4)
Deterministic reproducibility	Golden fixtures green on every commit — byte-identical, always
Decision capture completeness	100% of evaluated candles produce a snapshot row
Crash recovery	Auto-resume in correct state; zero orphaned trades in kill-tests
Live-vs-backtest divergence	Within tolerance on identical configs throughout the forward test

16.2 The Gates

Gate	When	Pass condition
G1	End wk 2	Foundation + replay harness deterministic on a sample month (Part 15.4)
G2	End wk 5	Engine complete offline; golden fixtures green; full lifecycle on replay
G3	End wk 7	GO/NO-GO. Walk-forward OOS results meet spec \$13 across ≥ 3 instruments / 3 years; sensitivity sweep stable. NO-GO = stop and rethink.
G4	End wk 11	Live infra: demo round-trip, kill-test, restore rehearsal all passed
G5	End wk 19	8-week forward test complete; divergence within tolerance; only then may the advisor influence live decisions
G6	Trade-count gated	Outcome learning shows held-out improvement; calibration holds on live outcomes

Hiring Roadmap — Triggered by Revenue, Not Hope

One founder plus Claude Code builds the system; people join when the P&L, not the plan, says so.

Trigger	Hire	Why then
Sustained live profitability over ≥ 2 quarters	Senior Python/quant engineer (part-time first)	Removes key-man risk on the core; the committed Graphify graph plus this blueprint is the onboarding pack
Capital or strategy count grows beyond one person's oversight	Operations / SRE (fractional)	24/7 watch, infra hygiene, broker relations
Research queue consistently outruns review capacity	Quant researcher	Stage-5 proposals deserve human depth; this hire mines the warehouse full-time
External capital or product offering	Compliance counsel (engagement, not employee)	The moment other people's money or promises enter, regulation does too

Each hire's first week: read this blueprint, query the graph, replay the golden fixtures. If the system can't be understood that way, fix the documentation before fixing the hiring.

The Honest Risk Register

Candour is a design principle. The biggest risks are listed first and stated plainly.

18.1 Strategy Risks

Risk	Likelihood	Mitigation
No real edge exists	High	Gate G3 at week 7: out-of-sample walk-forward before any infrastructure spend. Be willing to stop.
Overfitting — learning noise, not signal	High	Minimum samples, Bayesian shrinkage, mandatory OOS checks, sensitivity sweeps (Part 7)
Regime change decays the edge	Medium	Continuous capture + the learning loop detect decay; drawdown halts cap damage while you react
Costs erase a thin edge	Medium	Pessimistic fill model in every backtest; the R:R ≥ 2.5 floor gives margin
Synthetic-index quirks (Boom/Crash spikes)	Medium	Per-instrument configs; distinct ATR-EXTREME thresholds; never share parameters across instrument classes

18.2 Operational & Behavioural Risks

Risk	Likelihood	Mitigation
Founder overrides the rules under pressure	High	The state machine and halts are authoritative; manual intervention is logged and rare. No rule overridden by convenience.
Schedule pressure erodes the gates (a v2.1-specific risk)	Medium	The non-compressible floor (Part 13.2) is written down; phase locks are human-edited files; the forward test length is fixed in this document
Skipping analytics to "move faster"	Medium	Capture is architecturally mandatory; without it there is no Stage 3+
Single-person key-man risk	Medium	This document + the committed graph + reproducible infra; first senior hire when revenue allows
Warehouse loss	Low	Nightly encrypted off-site dumps; rehearsed restores. The one truly unrecoverable failure, treated accordingly.
Burnout from 24/7 operations	Medium	Strong alerting so the system runs unattended; halts and alerts make all-night vigilance unnecessary

THE QUANTUM REALITY CHECK

There is no quantum computer anywhere in this system and there does not need to be. The hypothesis engine is a classical Bayesian state model wearing borrowed vocabulary — useful internally as intuition, misleading externally if it implies exotic technology or guaranteed performance.

STANDING DISCLAIMER

Most retail algorithmic strategies show no durable edge. This blueprint is an engineering plan with honest gates, not investment advice and not a promise of profit. Its most valuable feature is the permission to stop at week 7.

Configuration Parameter Reference

Every tunable lives in `config/`, is hashed into `config_version`, and defaults to the `spec` value. The `spec` (v2.0 §Appendix) remains canonical for numbers; this table is the implementation index.

Parameter	Module	Notes
<code>ema_periods</code> = 21/50/89	M2	Fixed by spec; not a tuning surface
<code>atr_period</code> = 14; <code>atr_state_bounds</code>	M2/M6	Per-instrument; synthetics get distinct EXTREME thresholds
<code>sweep_score_min</code>	M5/M7	Gate-4 threshold; in the sensitivity sweep set
<code>rr_min</code> = 2.5	M8	Floor, never reduced by any advisor
<code>risk_pct_per_trade</code> ; <code>risk_tiers</code>	M8	Spec §risk; loss-streak state steps tiers down
<code>dd_daily_max</code> ; <code>dd_weekly_max</code>	M11	Breach → DAILY_HALT / weekly halt; adversarially tested
<code>loss_streak_threshold</code>	M11	Consecutive-loss state: risk down, operator alerted
<code>breakeven_at_r</code> = 1.0; <code>trail_from_r</code> = 1.5	M10	Trail anchors behind EMA21
<code>sessions_allowed</code> ; <code>news_tier1_buffer</code>	M12	Per instrument
<code>spread_max</code> ; <code>slippage_model</code> ; commission/swap	M9/replay	Pessimistic defaults; the backtest must not be kinder than the broker
<code>advisor_bounds</code> (min scale, veto floor)	Advisor	Stage 3+; advisor may act only within these
<code>min_samples_likelihoood</code> ; <code>min_trades_learning</code>	Stage 3/G6	Below them: shrink to no-evidence / gate stays closed
<code>walkforward</code> = 18m/6m/3m roll; ±20% sweep	Validation	v2.1 fixture of the G3 gate

Glossary

Term	Definition
Liquidity sweep	Price briefly trading through a level where resting orders cluster, then rejecting — the strategy's core event
Ascending triangle	Compression with roughly equal highs and rising lows — a bearish setup in this strategy
Descending triangle	Compression with roughly equal lows and falling highs — a bullish setup in this strategy
Sweep probability score	M5's 0–100 composite over five weighted factors: density, wick extension, close quality, EMA proximity, ATR state
R-multiple	Trade outcome in units of initial risk; the normalised number all learning trains on
Breakeven	Stop moved to entry at +1R; one-way once set
HALT state	New entries blocked by a risk trigger; open trades still managed
Advisor	A Stage 3–5 component behind a fixed interface returning confidence in [0,1] and an optional veto; can only reduce risk, never create a trade
Likelihood ratio	$P(\text{feature} \text{win}) \div P(\text{feature} \text{loss})$ — the atom of evidence, combined in log-odds
Calibration	Predicted probabilities match realised frequencies; verified out-of-sample before use
MarketStateVector	A probability distribution over mutually-exclusive regime hypotheses, updated each candle; its entropy measures market "indecision"
Walk-forward validation	Rolling train/test windows; verdicts rendered on out-of-sample segments only
Golden fixture	A committed input + expected decision log; CI fails on any byte difference — determinism as a test
Knowledge graph (Graphify)	Typed nodes/edges extracted from code, docs and schema in graphify-out/; queried instead of re-reading files
config_hash	sha256 of canonicalised parameters; stamps every row so any decision is reproducible years later